When the buyer isn't the user: modeling a dealer channel in HubSpot

A manufacturer sold through dealers but marketed to the operators who ran the machines. At the moment of sale the end customer vanished. Fixing it meant modeling the channel — not mapping more fields.

Paul Maxwell, CEO, RevOps HQ July 2026

ABSTRACT

An equipment manufacturer sells serial-numbered machines through a network of independent dealers, while its marketing speaks to the operators who actually use them. HubSpot's default contact–company–deal model assumes you sell to the buyer — but here the buyer (the dealer) and the customer (the operator) are different entities, and at the point of sale the end customer disappears. This is a first-principles look at modeling a channel business: representing dealer, employee, and end customer on one account without the association model collapsing; putting the install base in a custom object instead of a deal; recovering the end user through a warranty-capture loop; and using the Orders object as the bridge to the ERP.

1 CRMS ARE BUILT FOR DIRECT SALES

A CRM assumes the buyer is the customer: the company you sell to is the company you serve. Channel businesses break that assumption on contact one. The manufacturer's revenue flows through dealers, but the value — and the marketing — is aimed at the operator running the machine. Two different entities, and the default data model can only hold one.

The symptom shows up the instant a deal closes: HubSpot knows which dealer bought the unit and nothing about who runs it. Direct marketing, product-update outreach, and service all have nowhere to land, because the person they're for was never a record.

THE FIRST PRINCIPLE

Model the channel before you map fields. Decide who the buyer is, who the customer is, and how one account can hold a dealer, its employees, and its customers at once. Field mapping on top of the wrong entity model just moves the confusion faster.

2 WHERE THE END CUSTOMER DISAPPEARS

Follow one machine. The manufacturer sells it to a dealer; the dealer sells it to an operator. HubSpot records the dealer as the company and the sale as a deal — and stops there. The operator, the unit's location, its service history, and every future upsell live nowhere. Worse, when an operator does come in as a lead, associating them to the dealer makes them look like a dealer *employee*, not the dealer's *customer*. The relationship model quietly encodes the wrong thing.

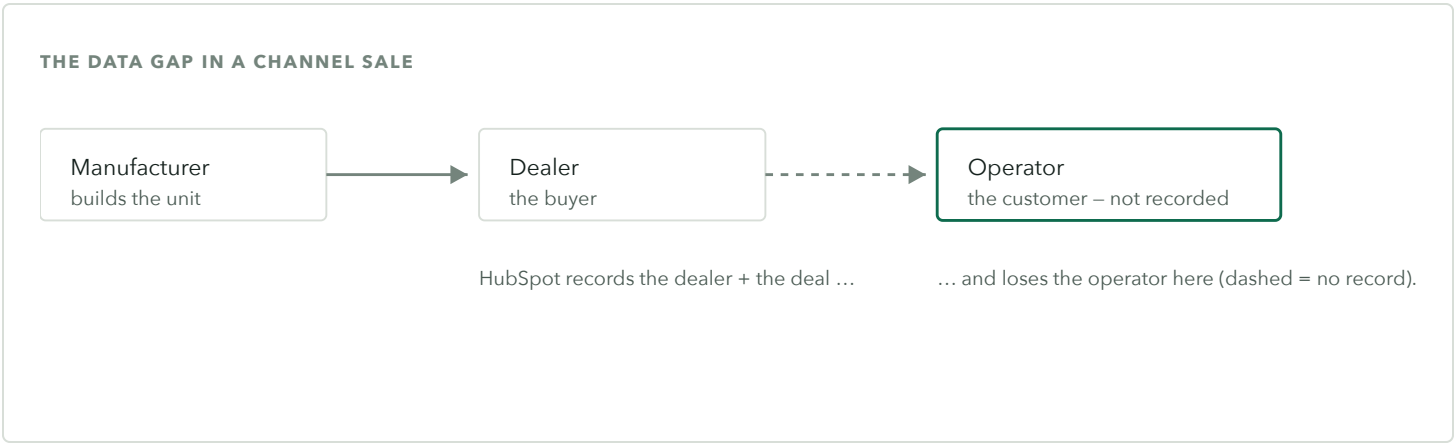

Figure 1. In a channel sale the CRM captures the dealer and the deal, but the operator – the person marketing and service actually care about – is never created. That gap is the whole problem.

3 ONE ACCOUNT, THREE ROLES

The fix starts with associations. A single company legitimately holds three kinds of people at once, and HubSpot can say so with **association labels**: *Dealer* (the channel relationship), *Employee* (the dealer's staff), and *Customer* (the operator the dealer sold to). A role property — owner, parts, sales, billing — adds the second dimension. Now a contact's place in the channel is data a workflow can read, not an assumption baked into a generic link.

WHY THIS MATTERS

Without labeled associations, every channel contact is an undifferentiated link and the model can't tell a dealer's employee from a dealer's customer. Labels are what let one account represent a real channel relationship.

4 THE INSTALL BASE IS AN OBJECT, NOT A DEAL

A sold machine is not a closed deal; it is a physical thing with a serial number, a location, an owner, and a service life. Modeled as a deal it disappears into the past the moment it's won. Modeled as an **Equipment custom object** — one record per unit — it becomes the spine of everything after the sale: which operator runs it, where, since when, and what service it's due. The unit associates to the selling dealer and, once known, to the operator.

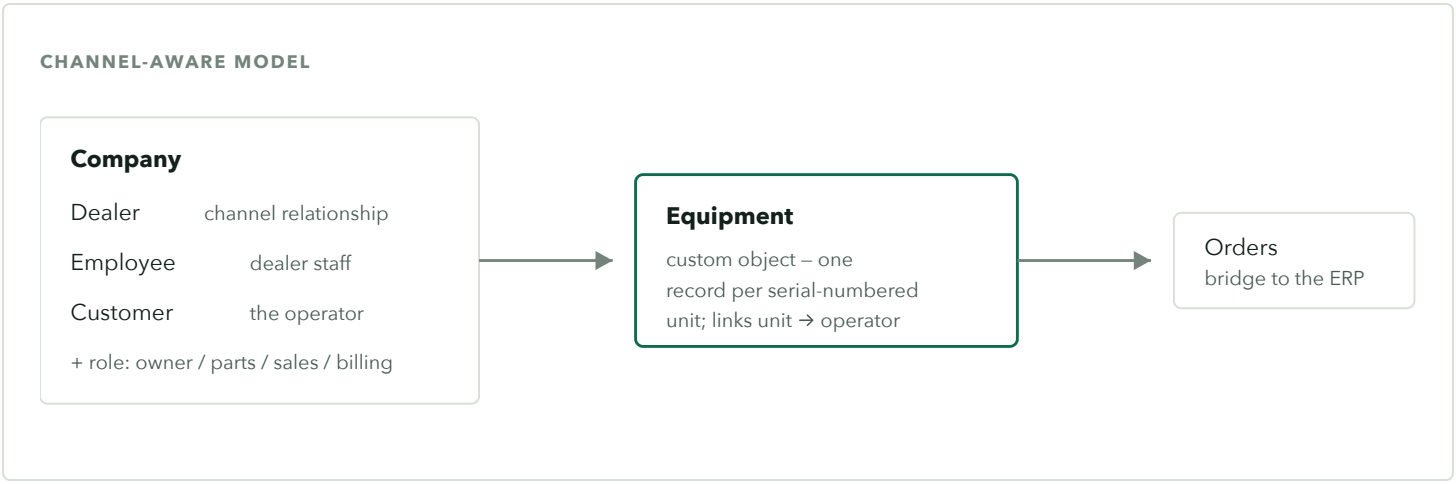

Figure 2. Association labels let one company hold dealer, employee, and customer roles; an Equipment custom object carries the install base and links each unit to its operator; the Orders object bridges to the ERP.

5 RECOVERING THE OPERATOR: A CAPTURE LOOP

Modeling the operator only helps if you can actually collect them. The recovery mechanism is a **warranty-registration loop**: a QR code or short URL on the paperwork that ships with every unit feeds a HubSpot form — operator, dealer, unit, serial number. Submitting it ties the operator to the machine and closes the data gap the channel opened. From that point the manufacturer can market to, update, and service the person who runs the equipment, not just the dealer who sold it.

6 THE ERP CATALOG ISN'T A SALES CATALOG

The manufacturer's products live in an ERP (NetSuite) — thousands of items, most of which a rep will never quote. Syncing them raw makes the HubSpot product picker unusable. The fix is a *quotable* flag on the ERP item, synced to a product property, so reps quote from a curated subset via native filtered product views. The **Orders** object maps one-to-one to ERP sales orders, and a light script pushes fulfillment and tracking back onto the order — so post-sale status lives on the record without HubSpot becoming the system of record for inventory.

> **TRANSFERABLE PATTERN**
>
> An ERP catalog is an inventory list, not a sales catalog. Sync a "quotable" flag and filter the product library on it, rather than dumping every SKU into HubSpot. Use the Orders object as the ERP bridge and keep inventory where it belongs.

7 WHAT CHANGED

The manufacturer can finally see past the dealer. The install base is a queryable set of records — which units are in the field, run by whom, due for what — instead of a pile of closed deals. Operators enter the system through warranty capture, so marketing and service reach the people who use the product. Reps quote from a clean, curated catalog, and orders carry their fulfillment status from the ERP. The channel is now something the CRM represents, not something it hides.

Details are generalized and figures are representative of the engagement; specifics vary by manufacturer.

8 THE MODEL, GENERALIZED

Any channel or install-base business faces the same exercise:

- **Separate buyer from customer.** If you don't sell to the end user, the CRM won't hold them unless you model it.
- **Use association labels** so one account can be dealer, employer, and seller-to-customer at once.
- **Put the install base in a custom object** — a sold unit is a thing with a life, not a closed deal.
- **Build a capture loop** (warranty, registration, activation) to recover the end user the channel hides.
- **Treat the ERP catalog as inventory,** not a sales catalog; filter to what's quotable and bridge orders.

RELATED STUDIES

The same "model before software" discipline runs through migrating an *advisory firm off a legacy CRM*, resolving *identity* where a native sync multiplies records, and modeling *project-based revenue* when the deal isn't the work.